Step 4 — Simulation Engine (Final Design)

Goal

The Simulation Engine controls the entire virtual broadcast world.

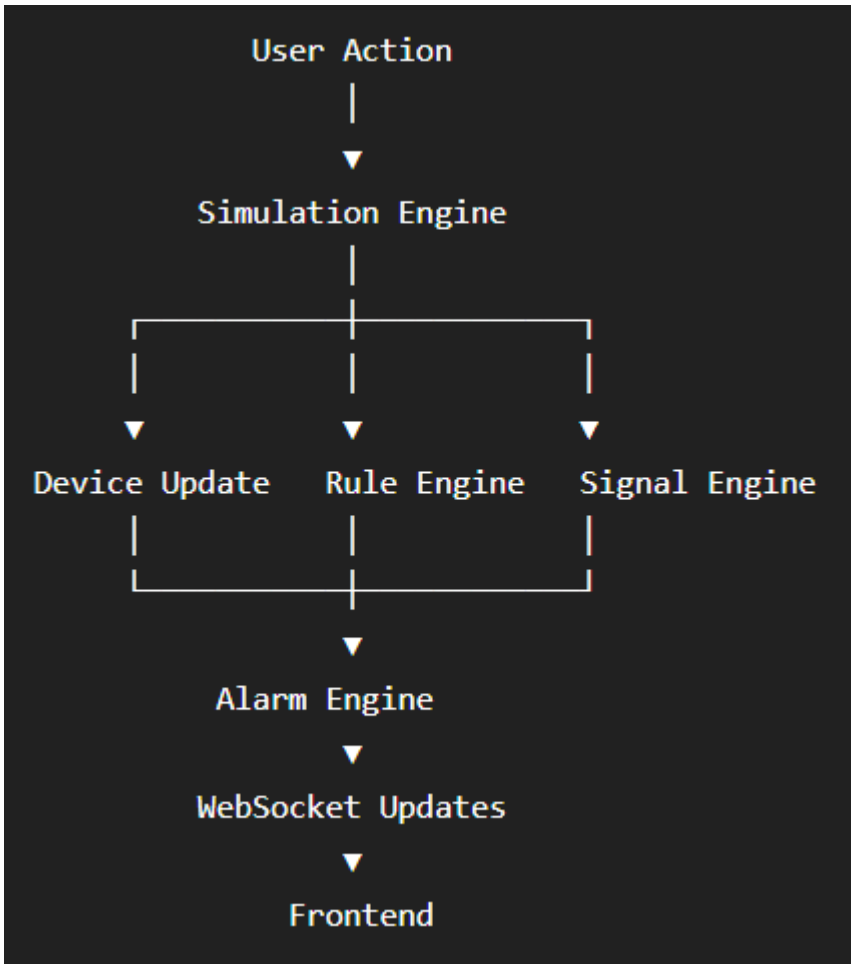
Nothing changes by itself.

Every device updates only when the Simulation Engine tells it to.

Think of it like Unity, Unreal Engine, or Cisco Packet Tracer.

---

Overall Flow




---

Simulation Tick

The engine runs continuously.

Example

Every 200 ms

↓

Tick 1

↓

Tick 2

↓

Tick 3

↓

Tick 4

Every tick represents time passing inside the simulator.

---

Simulation States

STOPPED

↓

RUNNING

↓

PAUSED

↓

STOPPED

Buttons

- Start
- Pause
- Resume
- Stop
- Reset

---

What Happens Every Tick?

The engine performs these steps in order.

1. Process User Events

↓

2. Update Devices

↓

3. Execute Rules

↓

4. Propagate Signals

↓

5. Detect Alarms

↓

6. Store Timeline

↓

7. Push WebSocket Updates

The order is important.

---

1. Process User Events

Example

User selects

Encoder

↓

High Temperature

Event Queue

HIGH_TEMPERATURE

Simulation Engine processes it first.

---

2. Update Devices

Every device receives

update()

The device

- updates timers
- updates metrics
- checks health

---

3. Rule Engine

Now calculate all dependent properties.

Example

FPS

30 → 60

↓

Bandwidth

↓

CPU

↓

Power

↓

Temperature

Nothing is random.

Everything follows rules.

---

4. Signal Propagation

Now the signal moves.

Example

Camera

↓

Router

↓

Encoder

↓

Viewer

If Camera disconnects

Router

↓

No Signal

↓

Encoder

↓

Black Frame

↓

Viewer

↓

No Video

The simulator behaves like a real broadcast chain.

---

5. Alarm Engine

Checks every device.

Example

Temperature

95°C

↓

Critical Alarm

or

CPU

95%

↓

Warning

---

6. Timeline

Every important action is recorded.

10:01 Camera Connected

10:03 Encoder Restarted

10:04 Temperature Warning

10:05 Viewer Lost Signal

---

7. WebSocket Updates

Only changed values are sent.

Example

Encoder

Temperature

55°C → 57°C

instead of sending the whole simulator state.

This is more efficient.

---

Simulation Clock

Instead of using real time directly, we'll introduce a Simulation Clock.

Real Time

↓

Simulation Time

This lets us add:

- Pause
- Resume
- Speed ×2
- Speed ×5

in the future without changing the engine.

---

Event Queue ⭐ (Addition)

Instead of immediately changing a device, every action goes into an Event Queue.

Example

Disconnect Camera

↓

Event Queue

↓

Simulation Tick

↓

Camera Disconnects

Why?

Because everything happens at the beginning of the next simulation tick.

This keeps the simulator deterministic and avoids inconsistent states.

---

Device Registry ⭐ (Addition)

The Simulation Engine shouldn't search for devices every tick.

Maintain a Device Registry.

Device Registry

Camera1

Camera2

Encoder1

Router1

The engine simply iterates through this collection.

---

Connection Registry ⭐ (Addition)

Similarly,

Connection Registry

Camera → Router

Router → Encoder

Encoder → Viewer

Signal propagation becomes efficient.

---

Simulation Context ⭐ (Addition)

Instead of passing many objects everywhere, create a single SimulationContext.

It contains references to:

Device Registry

Connection Registry

Simulation Clock

Rule Engine

Signal Engine

Alarm Engine

Timeline

Event Queue

Every module receives this context.

This keeps the architecture clean and avoids tight coupling.

---

Final Engine Architecture

Simulation Engine

|

├── Simulation Clock

├── Event Queue

├── Device Registry

├── Connection Registry

├── Rule Engine

├── Signal Engine

├── Alarm Engine

├── Timeline

└── WebSocket Publisher

---

Simulation Loop

while (RUNNING)

↓

Process Events

↓

Update Devices

↓

Apply Rules

↓

Propagate Signals

↓

Generate Alarms

↓

Log Timeline

↓

Publish Updates

↓

Wait for next Tick

---

Design Patterns Used

- Observer → Devices react to upstream signal changes.
- Strategy → Different calculation rules for different device types.
- Factory → Create devices from profiles.
- State → ONLINE, WARNING, FAILED, RECOVERING.
- Command → User events like Disconnect, Restart, Recover.
- Registry → Fast access to devices and connections.

---

Final Addition (Most Important)

I want one more concept that will make BroadcastSim much more realistic.

Simulation Modes

Instead of having only one behavior, support three modes:

1. Manual Mode

- Nothing changes automatically.
- Devices update only when the user edits properties or triggers events.
- Best for learning and demonstrations.

2. Automatic Mode

- The simulation engine updates metrics every tick.
- Devices heat up, cool down, consume power, and respond to load according to the rule engine.

3. Scenario Mode

- Load a predefined scenario (e.g., "Live Cricket Match", "News Studio").

- The engine injects scripted events over time, such as:
    - Camera disconnects at 2 minutes.
    - Encoder overheats at 5 minutes.
    - Router port fails at 7 minutes.
    - Recovery occurs at 8 minutes.

This makes BroadcastSim useful not just as a simulator but also as a training platform, where users can practice diagnosing broadcast faults under realistic conditions.